



Operating Systems

[1500WETOPS]

José Oramas



Virtual Memory

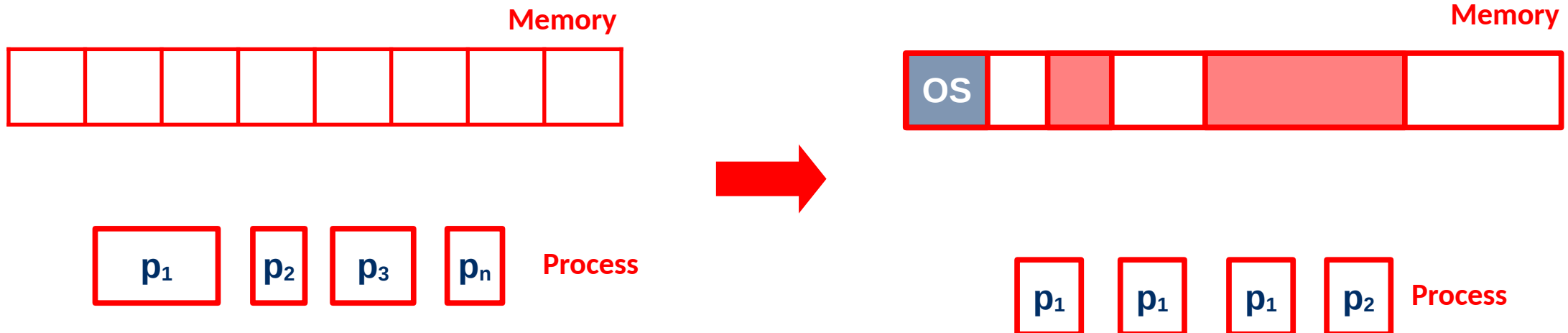
[Distributing process images accross multiple memory types]

José Oramas

Introduction

- **Memory Management**

→ the process image should not be stored as a single block



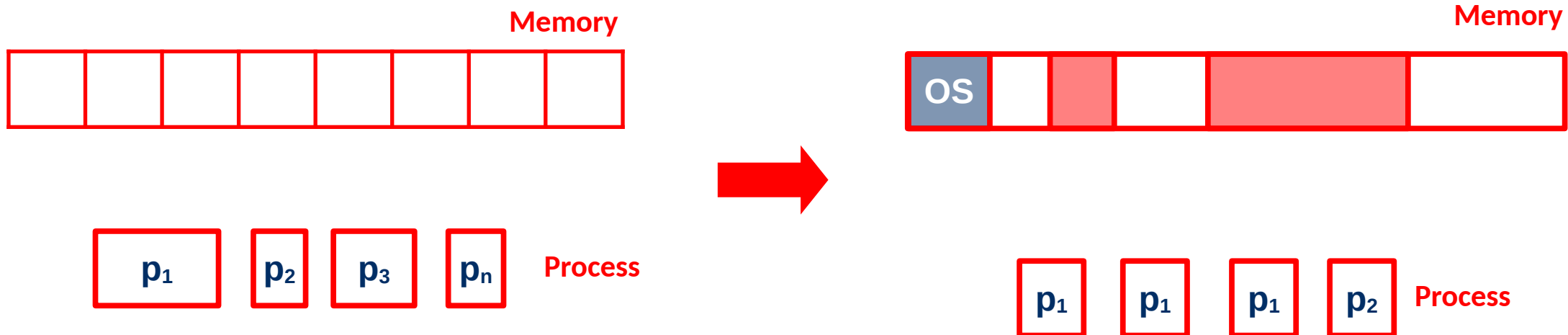
Introduction

- **Memory Management**

→ the process image should not be stored as a single block

- **Next Step**

→ All the pages from a process *do not* need to be in memory at the same time.



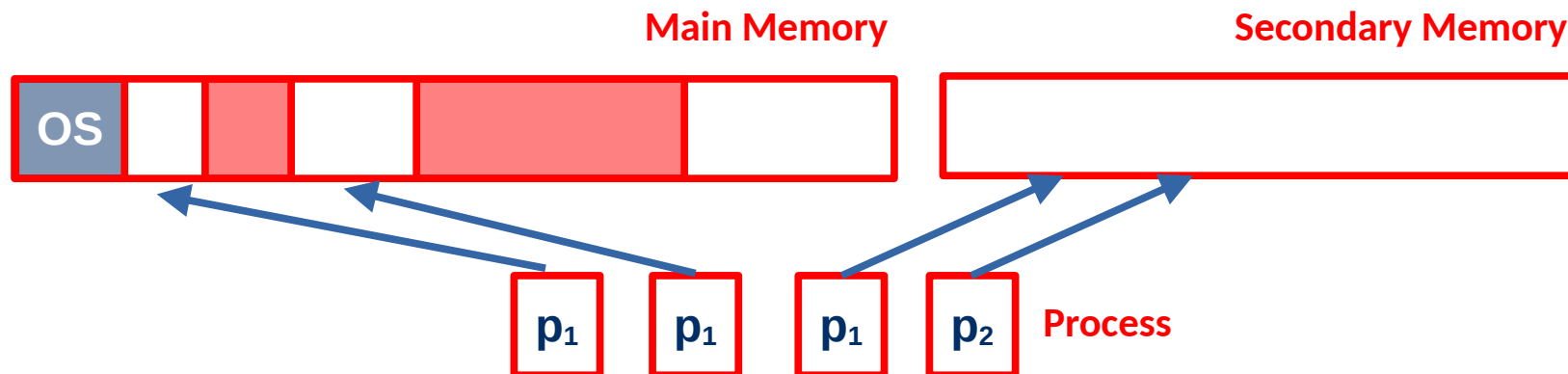
Introduction

- **Next Step**

→ All the pages from a process *do not* need to be in memory at the same time.

- **How?**

- *Load some of the pages in memory, keep the rest in swap space (disk)*
- Main memory < Swap space => logical address → virtual address



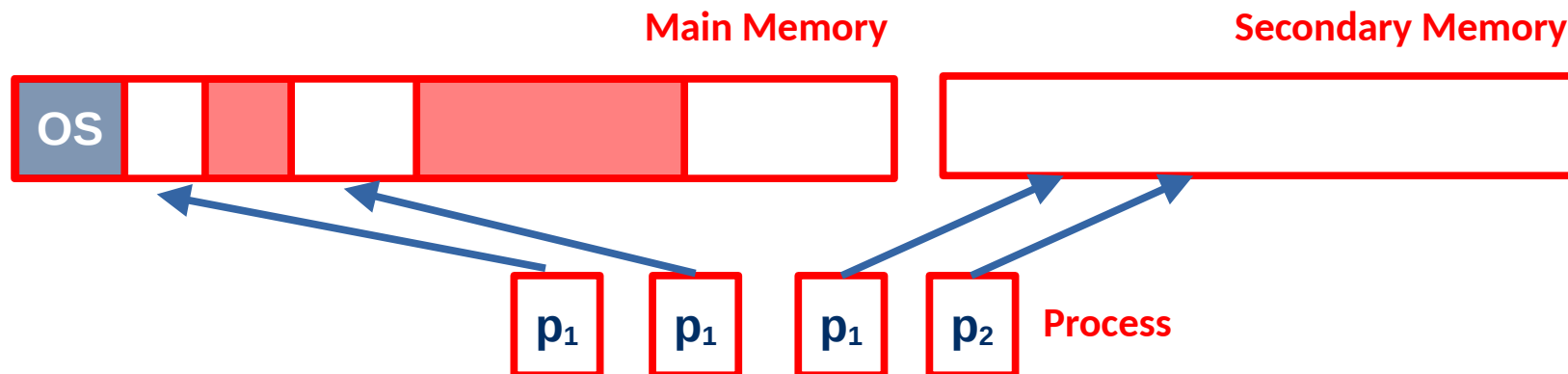
Introduction

- **Next Step**

→ All the pages from a process *do not* need to be in memory at the same time.

- **How?**

- *Load some of the pages in memory, keep the rest in swap space (disk)*
- Main memory < Swap space => logical address → virtual address



Any advantage?

Introduction

- **Similarities with Cache Memory**

- Cache Hit → Page Hit [Page is located in main memory]
- Cache Miss → Page Fault [handled by the OS]

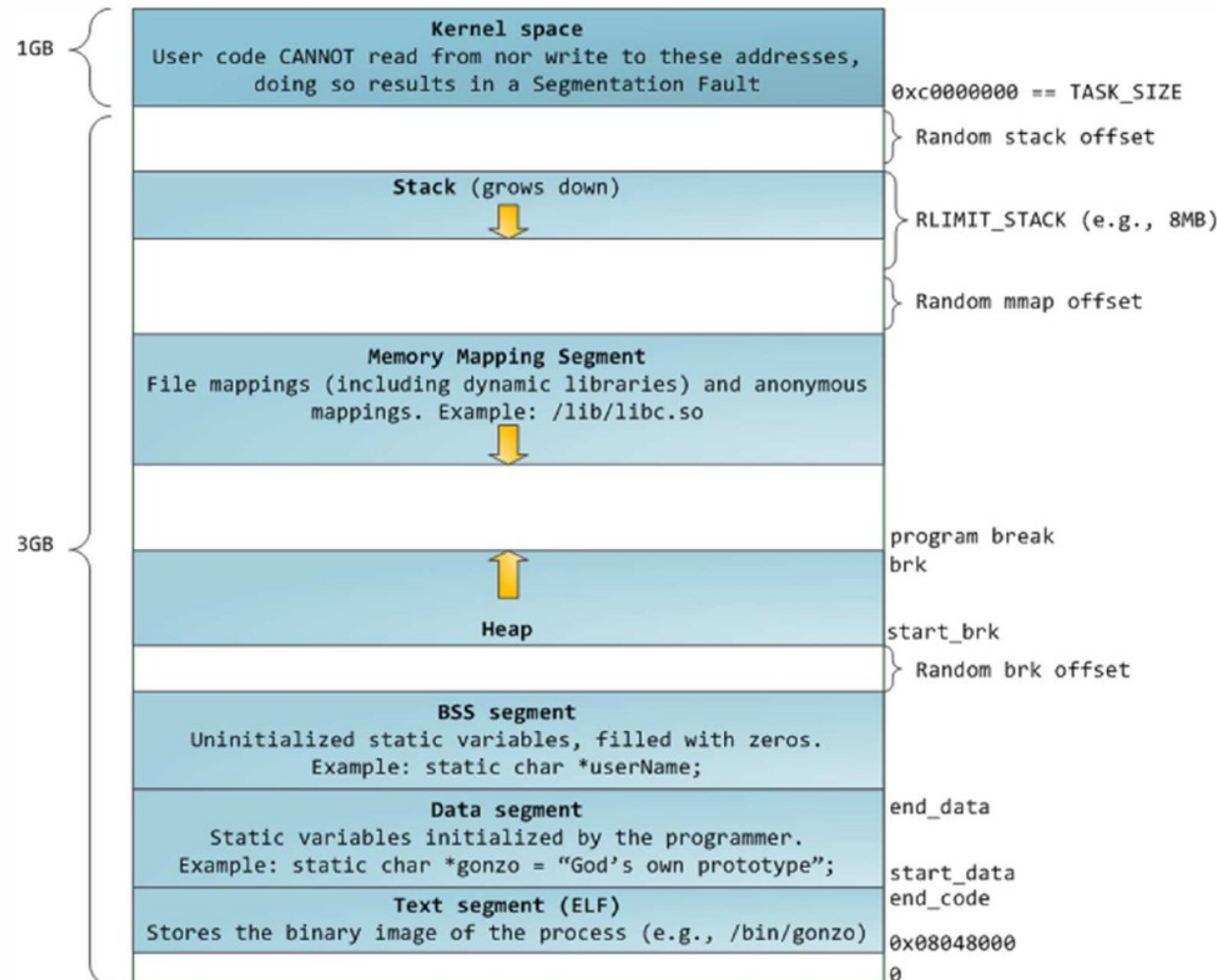
- **Page Search**

- Page search is done via the *page table*
- Each entry contains
 - *frame number* if present in memory

Virtual Memory Layout

Virtual Memory Layout

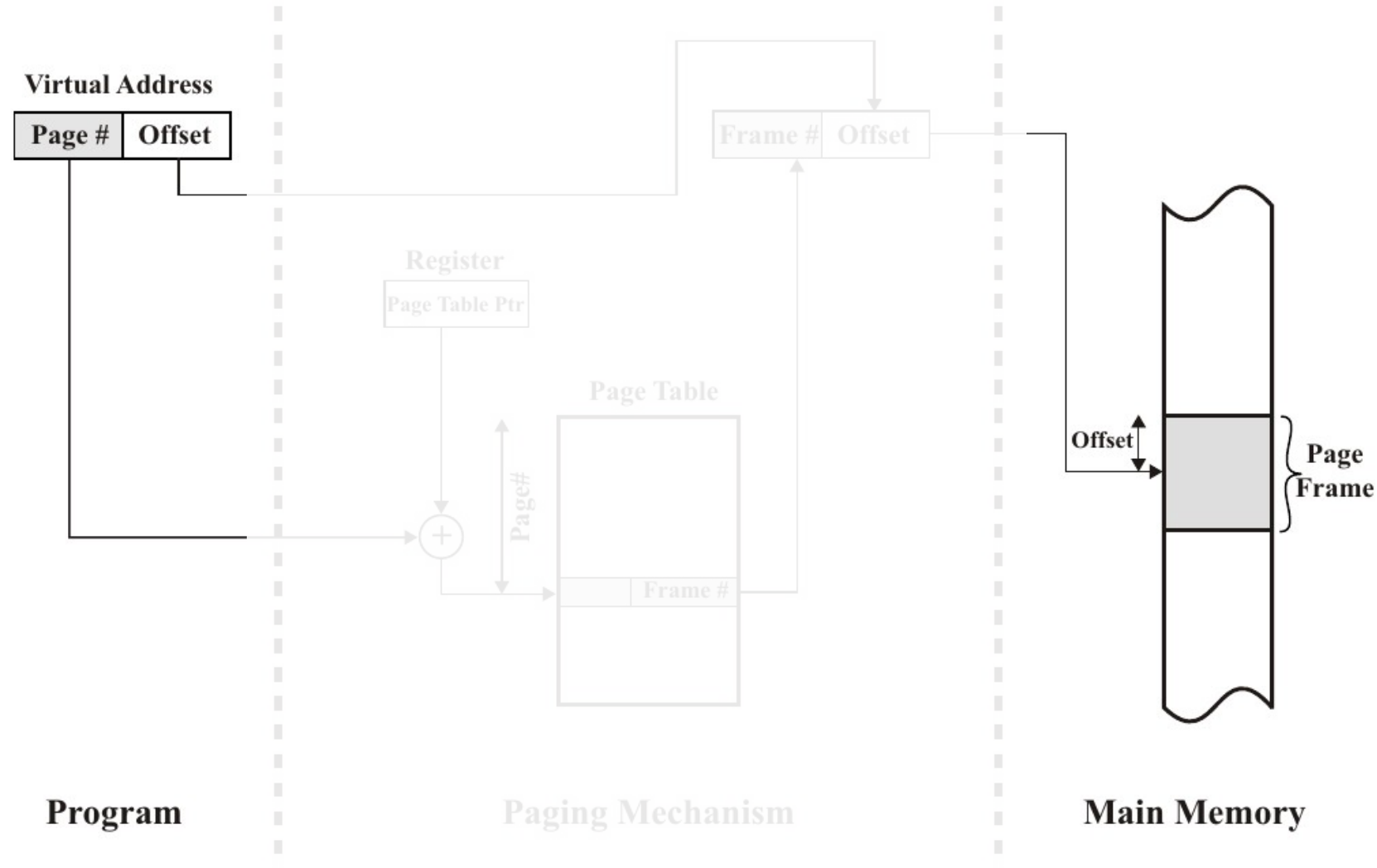
In a GNU-Linux on a 32-bit machine



If trying to do something not allowed to the segments → **segmentation fault**

Page Table Organization

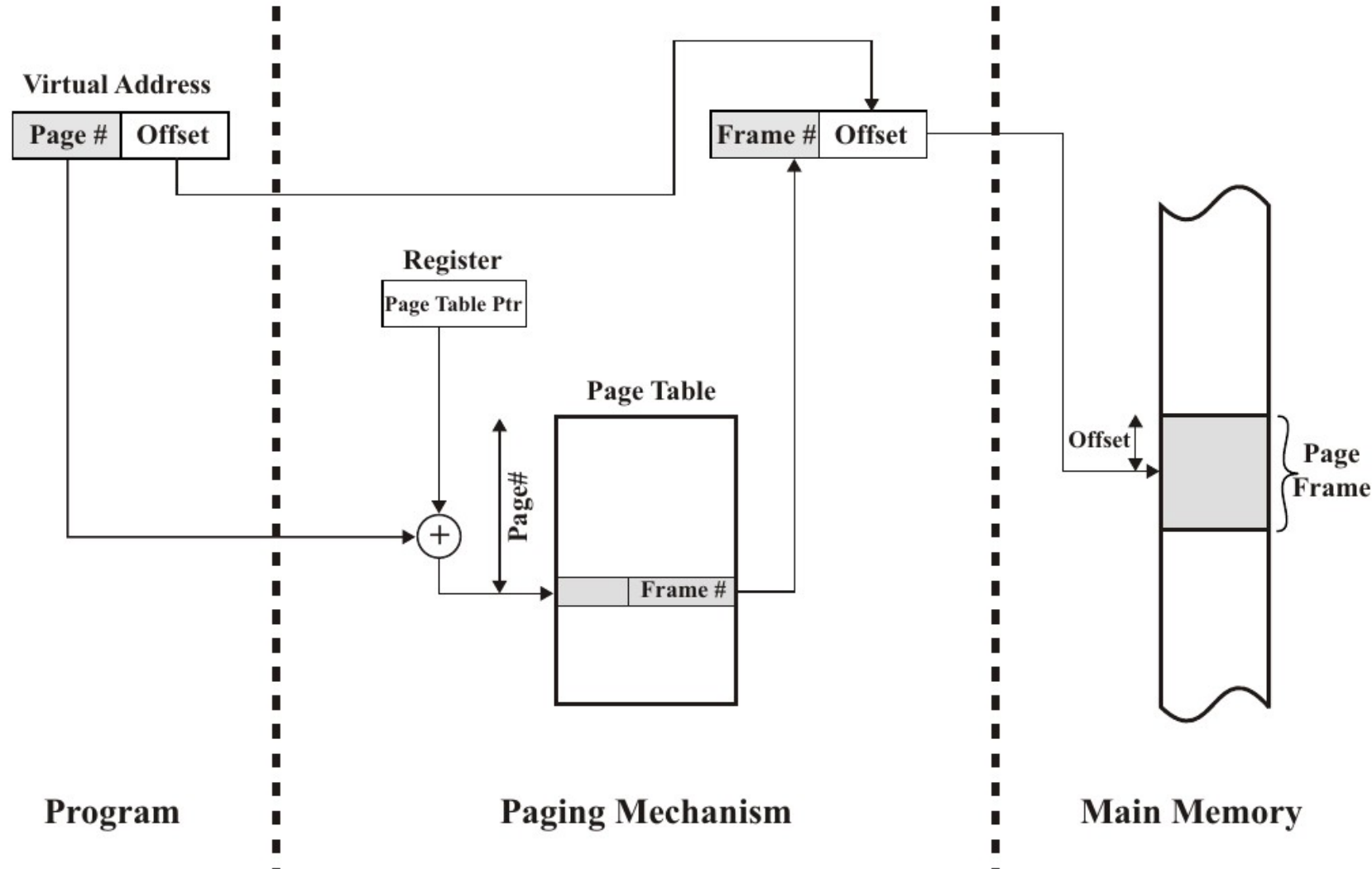
Page Table Organization



Page Table Organization

▪ Desirable

- Page table completely in memory
- Start address page table in register
- Page number as index to the table



Page Table Organization

■ Problem

- Page table itself is often very large
 - example: 4 GB virtual memory & 512 byte pages
⇒ 8M page entries (only partly used)
- Page number can no longer be an index

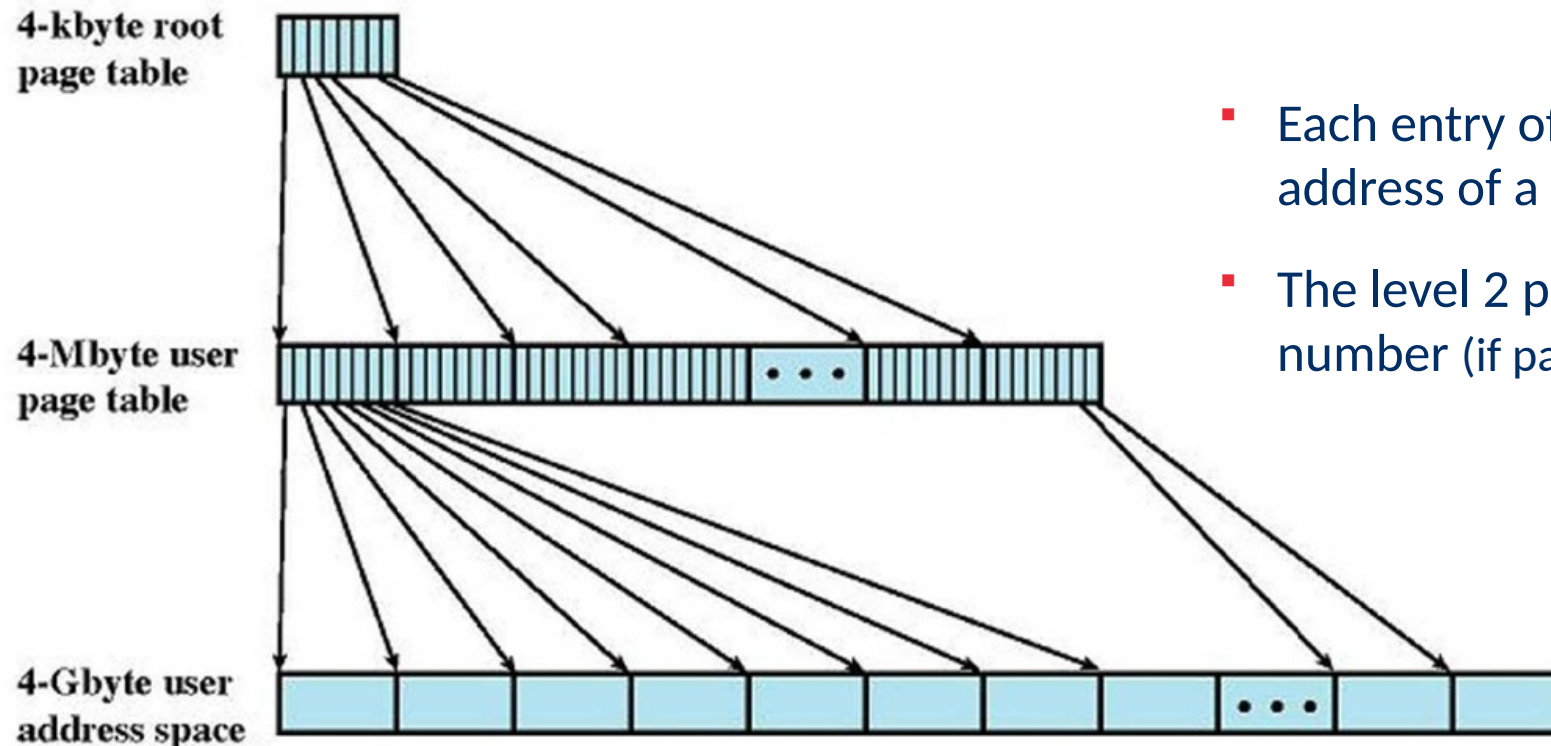
■ Solutions

- Multilevel page tables
- Inverted page tables
- etc.

Multilevel Page Tables

- Design

- First n bits of virtual page number (consisting of $n+m$ bits), as index to level 1 page table

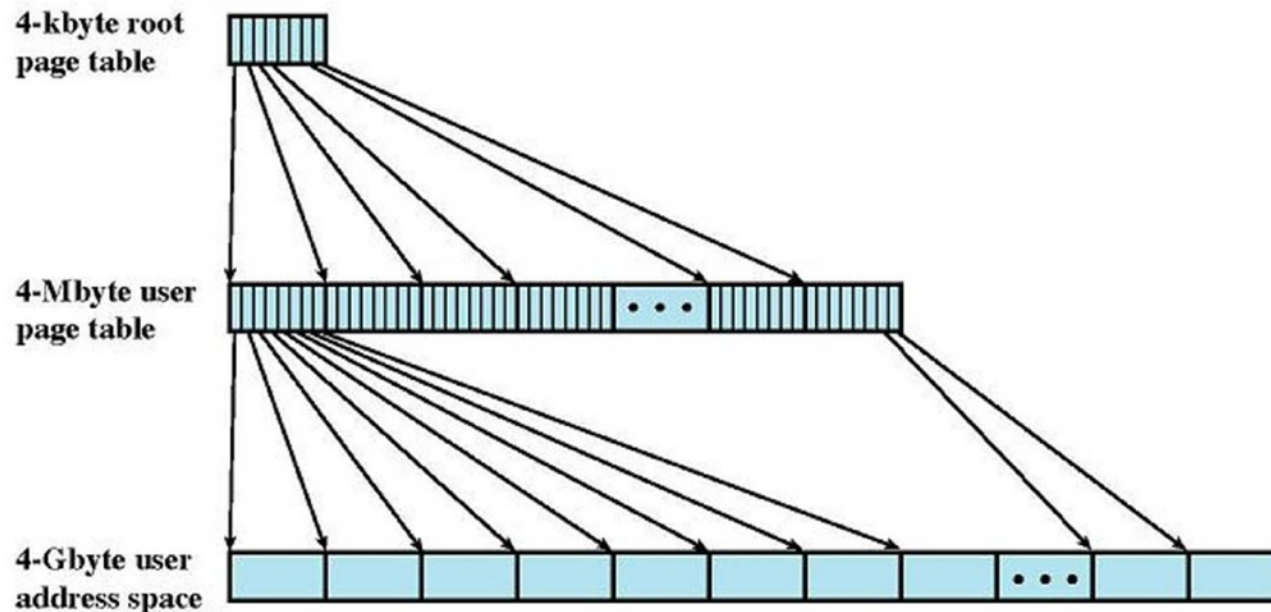


- Each entry of level 1 contains start address of a level 2 page table (size 2^m)
- The level 2 page table contains frame number (if page is in memory)

Multilevel Page Tables

- What is in main memory:
 - level 1 page table
 - the (recently) used level 2 page tables

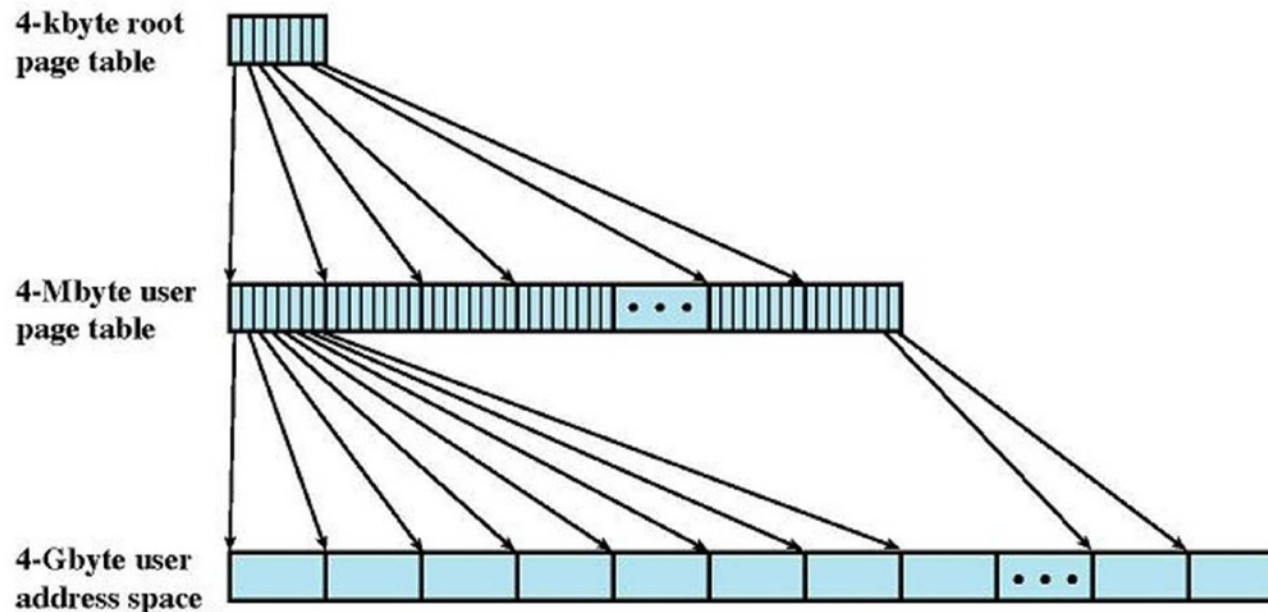
Any (dis)advantage?



Multilevel Page Tables

- What is in main memory:
 - level 1 page table
 - the (recently) used level 2 page tables

Any (dis)advantage?



Advantage:

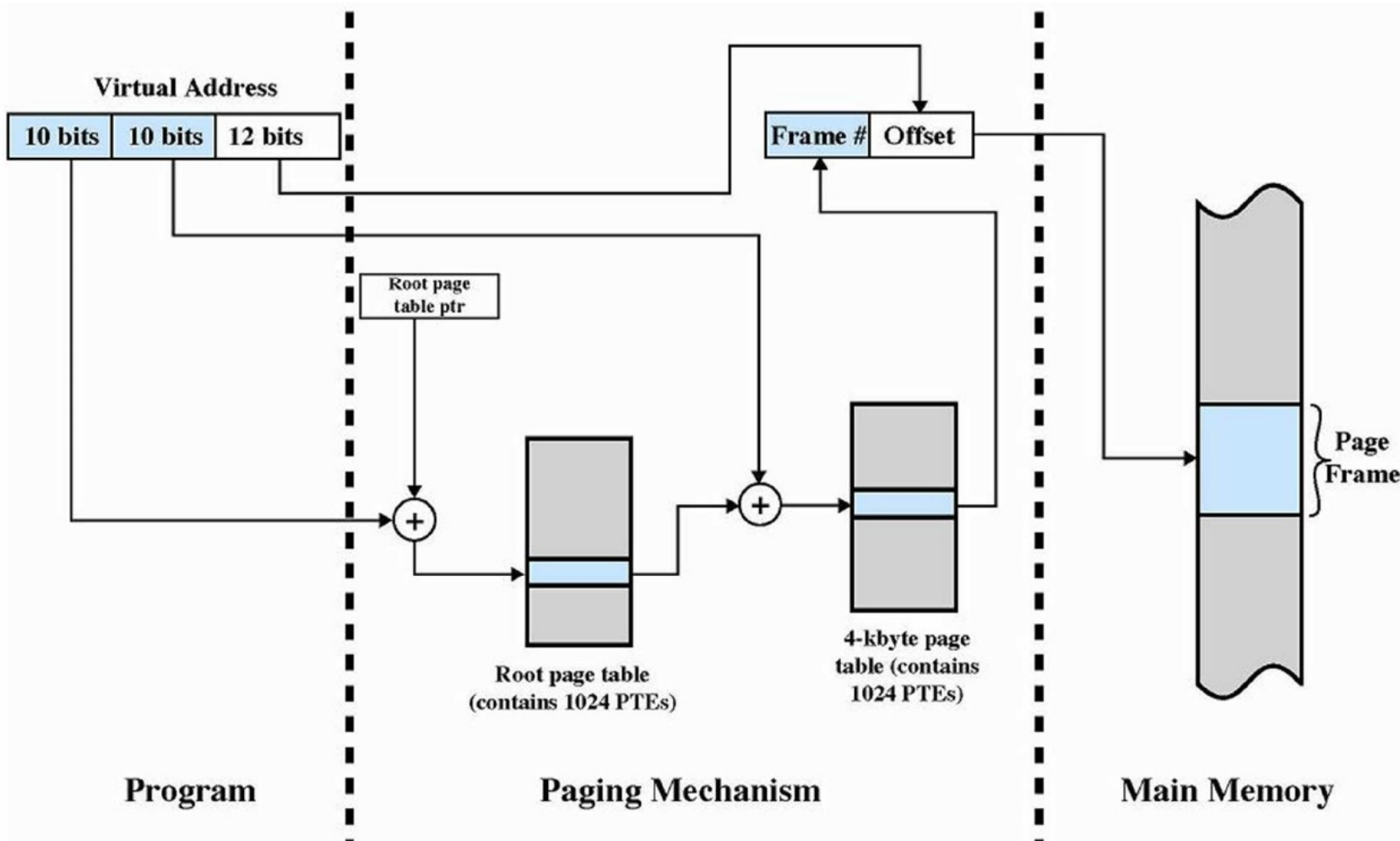
No need for level 2 tables for unused virtual memory areas

Disadvantage:

A lookout of a frame number requires multiple memory calls

Multilevel Page Tables

▪ 2-level Page Table - 32-bit x86 architecture



Composition

- 10 bits for level 1 index
- 10 bits for level 2 index
- 12 bits offset (4096 bytes pages)

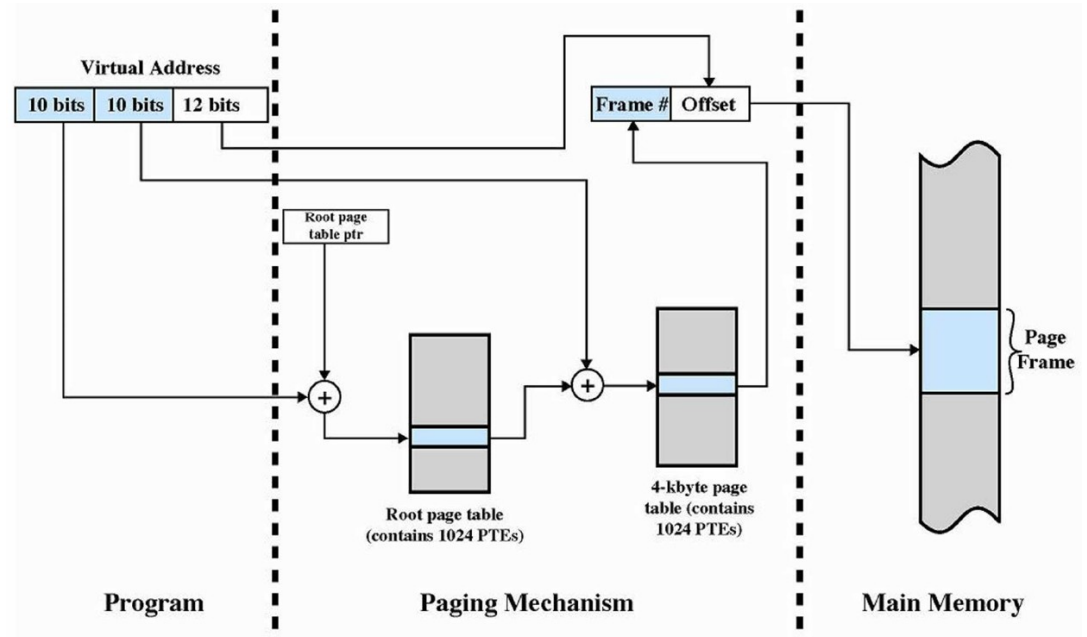
Note:

level 2 page table takes up exactly 1 page: start address in level 1 table entry = frame number of level 2 table

Multilevel Page Tables

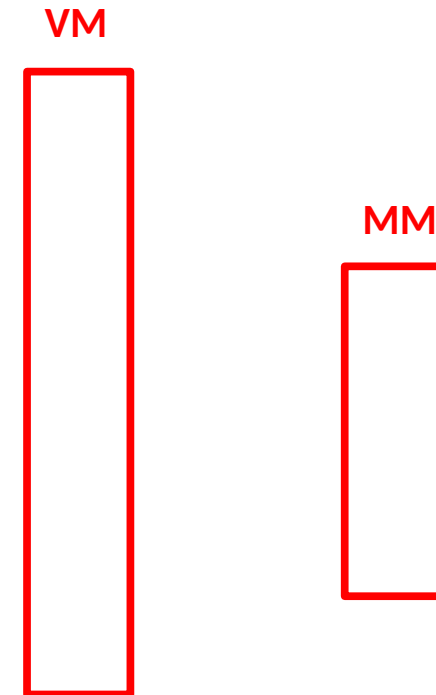
Possible Hit/Fault Scenarios

- Level 2 page table and requested page are in memory
→ no page faults
- Level 2 page table present, requested page not
→ 1 page fault
- Both missing
→ 2 page faults



Inverted Page Tables

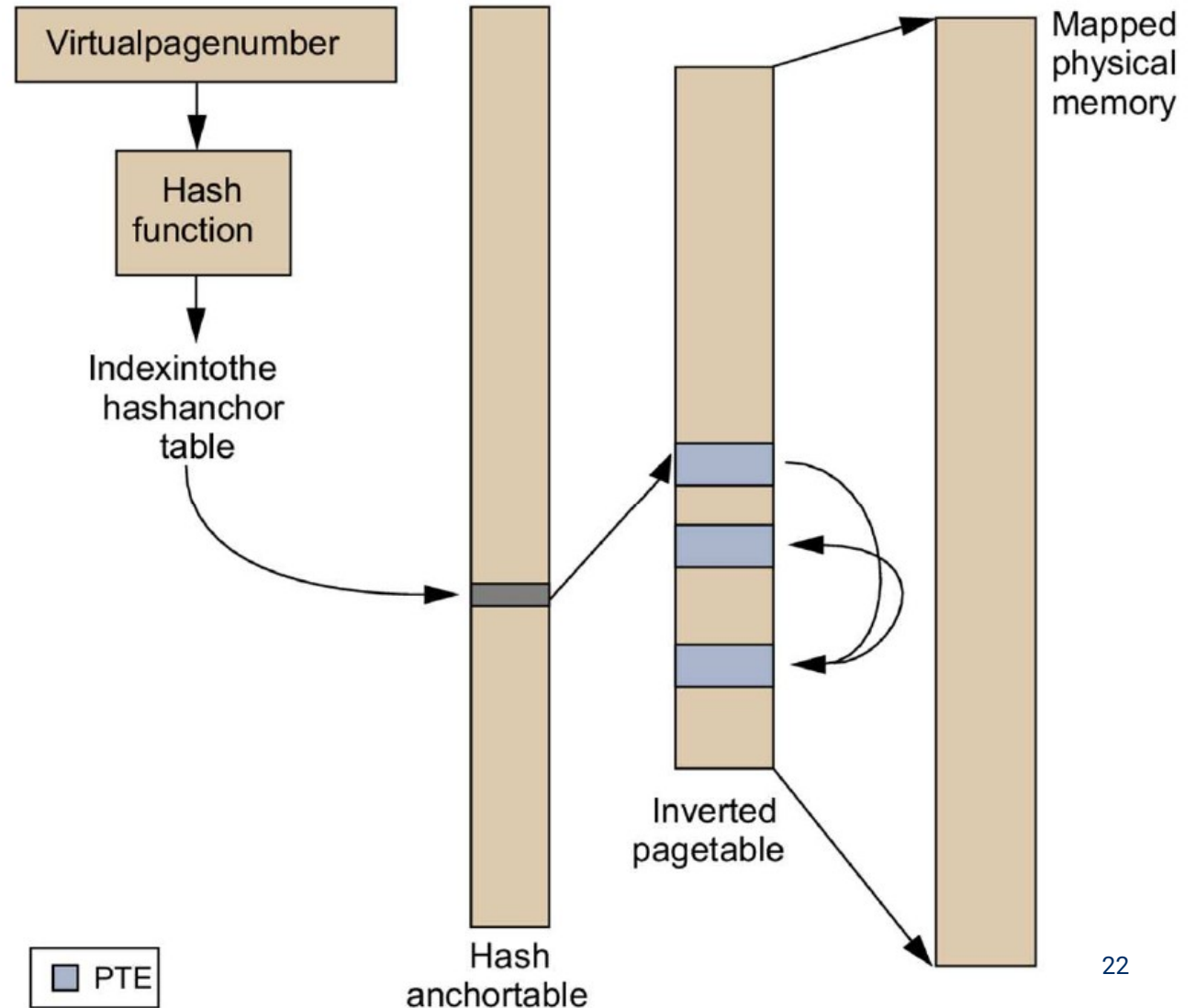
- **Problem:** There are more virtual pages than physical page frames
- **Idea:** table with an entry per page frame with (process-id, page number)
 - much smaller table
 - looking up is difficult
 - associative: very expensive
 - sequential: too slow
 - binary: expensive (to keep sorted)
- **Solution:** use hashing functions.



Inverted Page Tables

Hashing on Page Tables

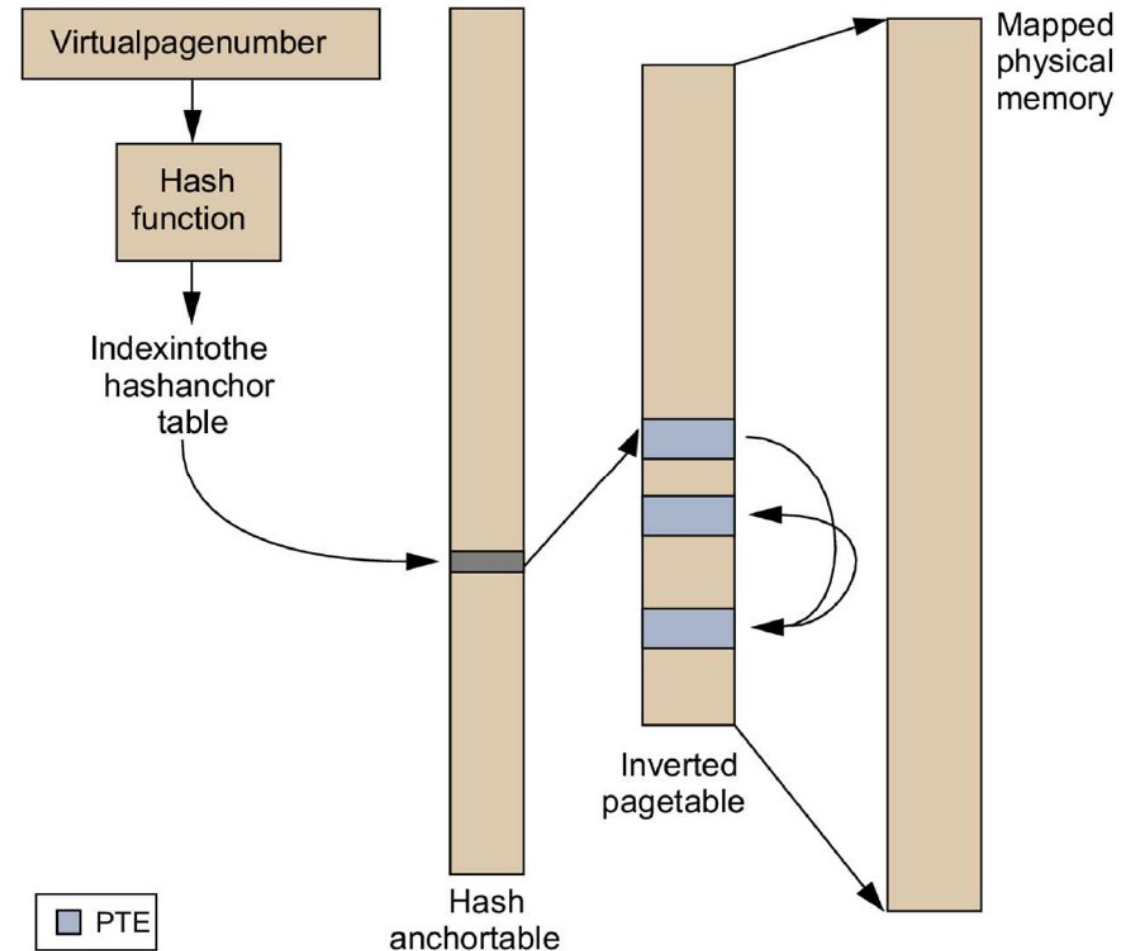
- Hash on (VPN+pid)
- Result is index to hash anchor table
- This refers to inverted page table with VPN numbers
- Entry k in the inverted page table contains the VPN of frame k
- Multiple (VPN+pid)s same hash result



Inverted Page Tables

Hashing on Page Tables

- Hash function
 - $M > N$, with M results, N frames
 - by anchor table, we can just use the index of the inverted table (when $M > N$)
 - good hash ensures that chains remain short
- Adding a new page to memory can easily be done via anchor table
- Sharing is more difficult: only one VPN per page table entry



Translation Lookaside Buffer (TLB)

TLB cache

- Reduces the number of times virtual-physical address translation occurs.
- Cache containing number (VPN, frame number) pairs
- Studied algorithms (direct, associative, set-associative) are applicable
- Always contains entries of process in execution
 - every process change should empty the TLB cache (TLB flush)

Virtual Memory: Software Support

Virtual Memory: Software Support

Remember

- Address translation is done by hardware
- If page fault → OS must ensure the right page is loaded in memory.

Virtual Memory: Software Support

Remember

- Address translation is done by hardware
- If page fault → OS must ensure the right page is loaded in memory.
- **Question:** in which frame to place the page to be loaded?
 - Q1: how to determine #pages per process?
 - Q2: can a page occupy the frame of another process?
 - Q3: how to choose the page to be replaced (from candidate frames)?

Virtual Memory: Software Support

Remember

- Address translation is done by hardware
- If page fault → OS must ensure the right page is loaded in memory.
- **Question:** in which frame to place the page to be loaded?
 - Q1: how to determine #pages per process?
 - Q2: can a page occupy the frame of another process?
 - Q3: how to choose the page to be replaced (from candidate frames)?
- Q1 & Q2 addressed via **resident set management**, Q3 via **replacement algorithms**.

Replacement Algorithms

Optimal Algorithm

- MIN algorithm
 - Replace page we are least likely to call back in the near future
- Properties
 - Guarantees minimum number of page faults
 - Requires perfect knowledge of all future event (not implementable in practice)
 - Used as reference

Replacement Algorithms

Least-Recently-Used (LRU) Algorithm

- LRU algorithm
 - Replace the least recently used page (~locality of reference)
- Properties
 - Stack implementation
 - Update of MRU position needed with each page hit
 - MRU position can become bottleneck (at threading)
 - No frequency information, sensitive to scan sequences

Replacement Algorithms

First-In-First-Out (FIFO) Algorithm

- FIFO algorithm
 - Replace the page that has been in memory the longest.
- Properties
 - Simple implementation → one pointer (circular buffer)
 - No update on hits → only works on page faults
 - Sensible to the Belady's anomaly

Replacement Algorithms

Belady's Anomaly

- For some page sequences, more memory will lead to more faults.

Page Requests	3	2	1	0	3	2	4	3	2	1	0	4
Frame 1	3	3	3	0	0	0	4	4	4	4	4	4
Frame 2		2	2	2	3	3	3	3	3	1	1	1
Frame 3			1	1	1	2	2	2	2	2	0	0

(red indicates page fault)

Replacement Algorithms

Belady's Anomaly

- For some page sequences, more memory will lead to more faults.

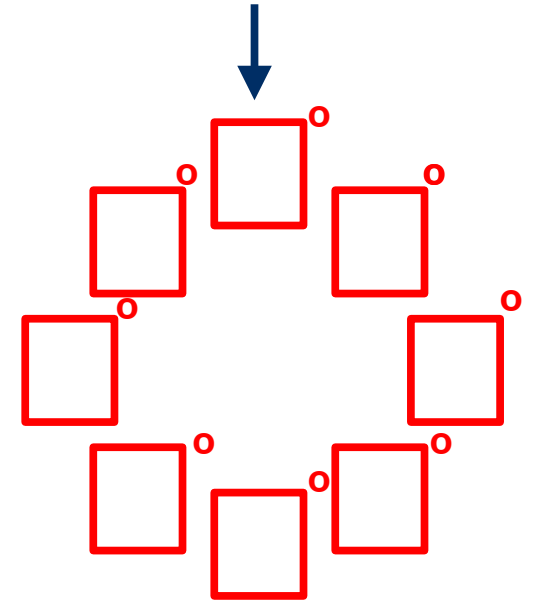
Page Requests	3	2	1	0	3	2	4	3	2	1	0	4
Frame 1	3	3	3	0	0	0	4	4	4	4	4	4
Frame 2		2	2	2	3	3	3	3	3	1	1	1
Frame 3			1	1	1	2	2	2	2	2	0	0
Page Requests	3	2	1	0	3	2	4	3	2	1	0	4
Frame 1	3	3	3	3	3	3	4	4	4	4	0	0
Frame 2		2	2	2	2	2	2	3	3	3	3	4
Frame 3			1	1	1	1	1	1	2	2	2	2
Frame 4				0	0	0	0	0	0	1	1	1

(red indicates page fault)

Replacement Algorithms

Clock Algorithm

- Frames form circular buffer with *use bit* per frame
- On page hit, set *use bit* = 1
- On page fault, look for the next frame with *use bit* = 0,
 - set all *use bits* we pass = 0
- Properties
 - between LRU and FIFO
 - Sometimes the whole list is run through
 - frequency less important if list is long



Replacement Algorithms

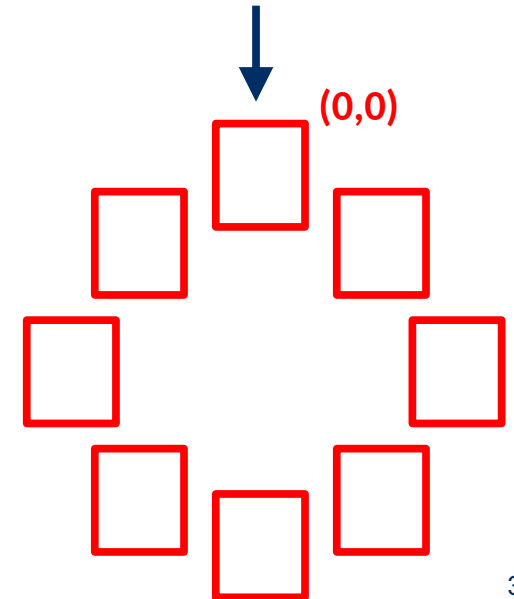
Clock Algorithm - Variant

- Update bit

- Look for locations $(0,0)$ with both *use bit* == 0 and *update bit* == 0,
- If not found, look for location $(0,1)$ with *use bit* == 0, and *update bit* == 1 (set use bits we pass to 0)
- If not found, repeat the two steps again (it will now succeed) .

- Properties

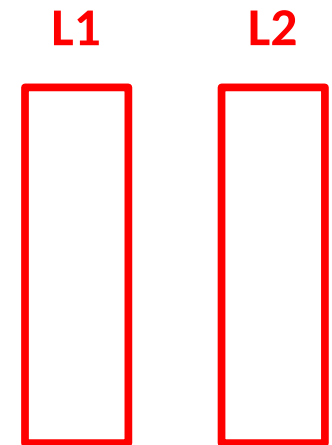
- Aims at replacing unchanged frames first ← why?



Replacement Algorithms

Page Buffering & Segmented FIFO

- Segmented FIFO
 - 2 buffers: **L1** = FIFO, **L2** = 'LRU'
 - Frame falls out of L1, goes to top of L2
 - Frame falls out of L2, released from memory
 - L1 hit: do nothing
 - L2 hit: move frame to L1
- Properties
 - between FIFO and LRU (adjustable via size L1 & L2)



Resident Set Policy

Goal: determines which frames are eligible for replacement.

- Some frames are never eligible (examples?)
- Fixed vs. Variable allocation
 - # of page frames per process.
- Replacement Policy
 - Local: pages can only claim their own page frames
 - Global: pages can claim their page frames from other processes
- Active Working Set – $W(t, \Delta)$
 - at time t , all pages requested by a process during the last Δ seconds

Resident Set Policy

Active Working Set – $W(t, \Delta)$

- **Question:** How to determine the number of pages dynamically?
- At time t , all pages requested by a process during the last Δ seconds
 - Characterized by periods of stability and transition periods
 - Monitored via page fault rate/interval
 - Indicate whether W is larger than total available frames
- how to limit the set at transition moments?

Key points

[Take Home Message]

Virtual Memory

Pay attention to...

- How can we organize page tables efficiently?
- The complete procedure to be followed when querying a memory word in a virtual memory system
- The main page replacement algorithms and their advantages and disadvantages?

Further Reading

Essential: Virtual Memory

- OS Course Notes - Chapter 4.
- Blackboard – Recorded lecture from Prof. Van Houdt.
[Special attention to the proof related to the MIN algorithm]

Exercise Session 28/10/2025

- Memory Management & Virtual Memory

Questions?



Operating Systems

[1500WETOPS]

José Oramas